Advanced Machine Learning

Lecture 10: Residual Neural Networks & Variants

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

17 December 2024

Outline

Outline

Long-term dependencies

Simple RNN suffer from one major problem: that of long-term dependencies.

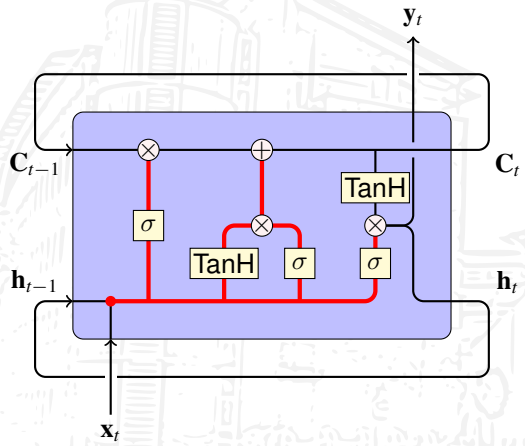
As the hidden state is transformed in every step by a matrix multiplication, long-term dependencies are hard to learn. The classical example is a sentence like

> *I was born in France. [. . .] I speak French.*

The longer the part in brackets, the smaller the probality that the RNN learns this long-term dependency. RNN can only learn dependencies that are as long ago as the unrolling parameter permits. In many cases RNN do not learn dependencies that are too long-term; simple RNN only have a short-term memory.

The solution to deal with long-term dependencies are Long Short-Term Memory (LSTM) networks. These have been published in Hochreiter and Schmidhuber [5].

LSTM



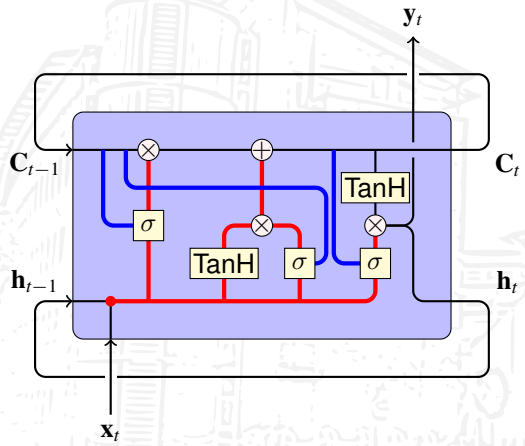
LSTM have a hidden state $\mathbf{h}_t$ and an additional cell state $\mathbf{C}_t$ that stores long-term dependencies.

The inner structure is more complicated than for a simple RNN, we have

- five classical layers w/ weight matrices, biases, thrice the logistic function σ, two activation functions, here twice TanH,
- three componentwise multiplications and one componentwise addition,
- one vector concatenation, five copies.

In this example the output $\mathbf{y}_t$ is equal to the hidden state $\mathbf{h}_t$.

LSTM variants: peephole connections



There exist variants that introduce peepholes that allow the gates to see the cell state, e.g., the forget gate looks as follows:

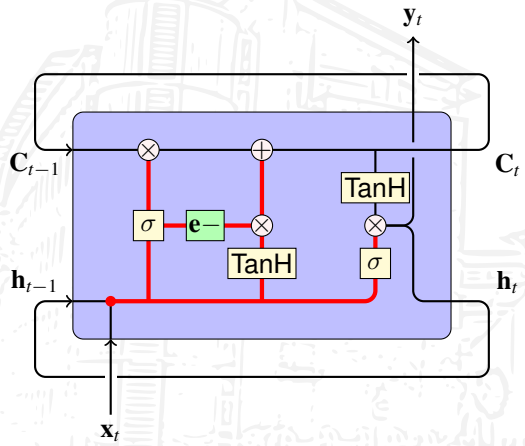
$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \begin{pmatrix} \mathbf{C}_{t-1} \\ \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} + \mathbf{b}_f\right),$$

where $\mathbf{W}_f$ splits into

$$\mathbf{W}_f = \begin{pmatrix} \mathbf{W}_{fC} & \mathbf{W}_{fh} & \mathbf{W}_{fx} \end{pmatrix}.$$

This is a generalization: In case $\mathbf{W}_{fC} = \mathbf{O}$ we recover the variant w/o peephole.

LSTM variants: coupled input and forget gates



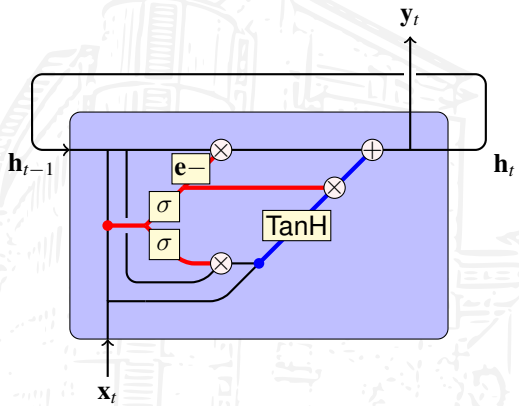
Using a model with limited memory, we insert something new when we forget something. Thus, a variation on LSTM is to couple the input and forget gates,

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \widetilde{\mathbf{C}}_t \circ (\mathbf{e} - \mathbf{f}_t),$$

where $\mathbf{e}$ is the vector of all ones.

Here, $\mathbf{i}_t = \mathbf{e} - \mathbf{f}_t$ and we compute the new cell state as a convex combination of the old cell state and the new candidate cell state.

LSTM variants: GRU



The Gated Recurrent Unit (GRU) by Cho et al. [1] is a more dramatic variation of an LSTM.

Here, cell and hidden state are merged, input and forget gate are coupled, and no bias is used,

$$\mathbf{z}_t = \sigma\left(\mathbf{W}_z \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix}\right), \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix}\right),$$

$$\widetilde{\mathbf{h}}_t = \text{TanH}\left(\mathbf{W}_h \begin{pmatrix} \mathbf{h}_{t-1} \circ \mathbf{r}_t \\ \mathbf{x}_t \end{pmatrix}\right),$$

$$\mathbf{h}_t = (\mathbf{e} - \mathbf{z}_t) \circ \mathbf{h}_{t-1} + \mathbf{z}_t \circ \widetilde{\mathbf{h}}_t.$$

Only two gates: update gate $\mathbf{z}_t$ & reset gate $\mathbf{r}_t$.

BPTT for LSTM

We consider the defining equations of an LSTM and derive the corresponding backpropagation through time.

The LSTM is based on the three gates (forget gate, input gate, output gate),

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \begin{pmatrix}\mathbf{h}_{t-1}\\ \mathbf{x}_t\end{pmatrix} + \mathbf{b}_f\right), \quad \mathbf{i}_t = \sigma\left(\mathbf{W}_i \begin{pmatrix}\mathbf{h}_{t-1}\\ \mathbf{x}_t\end{pmatrix} + \mathbf{b}_i\right), \quad \mathbf{m}_t = \sigma\left(\mathbf{W}_h \begin{pmatrix}\mathbf{h}_{t-1}\\ \mathbf{x}_t\end{pmatrix} + \mathbf{b}_h\right),$$

the cell state update

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \widetilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \widetilde{\mathbf{C}}_t = \mathrm{TanH}\left(\mathbf{W}_C \begin{pmatrix}\mathbf{h}_{t-1}\\ \mathbf{x}_t\end{pmatrix} + \mathbf{b}_C\right),$$

and the hidden state update that depends on the previous hidden state,

$$\mathbf{h}_t = \mathrm{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

BPTT for LSTM

Like in the simple RNN case we consider a cost function, here $\mathbf{y}_t = \mathbf{h}_t$,

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^{k} C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

The hidden state evolves as follows,

$$\mathbf{h}_t = \mathrm{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t = f(\mathbf{h}_{t-1}).$$

For the last hidden state in step $t = k$ we can easily compute

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k},$$

for the previous steps $t < k$ we end up with the recursion

$$\frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial C_t}{\partial \mathbf{h}_t} + \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \cdot \frac{\partial C}{\partial \mathbf{h}_{t+1}}.$$

BPTT for LSTM

A similar recursion develops for the cell state $\mathbf{C}_t$, which occurs in the two formulas

$$\mathbf{C}_{t+1} = \mathbf{C}_t \circ \mathbf{f}_{t+1} + \widetilde{\mathbf{C}}_{t+1} \circ \mathbf{i}_{t+1}, \quad \mathbf{h}_t = \mathsf{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t.$$

In the last step $t = k$ we only have to compute

$$\frac{\partial C}{\partial \mathbf{C}_k} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{C}_k} \cdot \frac{\partial C}{\partial \mathbf{h}_k} = \Big(\mathbf{e} - \mathsf{TanH}^2(\mathbf{C}_k)\Big) \circ \mathbf{m}_k \circ \frac{\partial C}{\partial \mathbf{h}_k},$$

in previous steps $t < k$ we have the recurrence

$$\frac{\partial C}{\partial \mathbf{C}_t} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{C}_t} \cdot \frac{\partial C}{\partial \mathbf{h}_t} + \frac{\partial \mathbf{C}_{t+1}}{\partial \mathbf{C}_t} \frac{\partial C}{\partial \mathbf{C}_{t+1}} = \Big(\mathbf{e} - \mathsf{TanH}^2(\mathbf{C}_t)\Big) \circ \mathbf{m}_t \circ \frac{\partial C}{\partial \mathbf{h}_t} + \mathbf{f}_{t+1} \circ \frac{\partial C}{\partial \mathbf{C}_{t+1}}.$$

Thus, in an LSTM, we have two recurrences, one for the hidden state, one for the cell state.

BPTT for LSTM

We define

$$\mathbf{a}_t := \begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{m}_t \\ \widetilde{\mathbf{C}}_t \end{pmatrix}, \quad \mathbf{z}_t := \begin{pmatrix} \mathbf{z}_{ft} \\ \mathbf{z}_{it} \\ \mathbf{z}_{ht} \\ \mathbf{z}_{Ct} \end{pmatrix} := \begin{pmatrix} \mathbf{W}_{fh}\,\mathbf{h}_{t-1} + \mathbf{W}_{fx}\,\mathbf{x}_t + \mathbf{b}_f \\ \mathbf{W}_{ih}\,\mathbf{h}_{t-1} + \mathbf{W}_{ix}\,\mathbf{x}_t + \mathbf{b}_i \\ \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{hx}\,\mathbf{x}_t + \mathbf{b}_h \\ \mathbf{W}_{Ch}\mathbf{h}_{t-1} + \mathbf{W}_{Cx}\mathbf{x}_t + \mathbf{b}_C \end{pmatrix} =: \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t + \mathbf{b}.$$

We have (thrice logistic function, once hyperbolic tangent)

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{z}_t} = \operatorname{diag} \begin{pmatrix} (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \\ (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \\ (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \\ \mathbf{e} - \widetilde{\mathbf{C}}_t \circ \widetilde{\mathbf{C}}_t \end{pmatrix}, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_h^\mathsf{T}, \quad \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} = \mathbf{W}_x^\mathsf{T}.$$

On the next slide we consider the three gates and the candidate state by looking at the partial derivatives with respect to $\mathbf{z}_{ft}$, $\mathbf{z}_{it}$, $\mathbf{z}_{ht}$, and $\mathbf{z}_{Ct}$.

BPTT for LSTM

We use

$$\mathbf{C}_t = \mathbf{C}_{t-1} \circ \mathbf{f}_t + \widetilde{\mathbf{C}}_t \circ \mathbf{i}_t, \quad \mathbf{h}_t = \mathsf{TanH}(\mathbf{C}_t) \circ \mathbf{m}_t$$

to compute the derivatives with respect to the gates and candidate cell state,

$$\begin{aligned}
\frac{\partial C}{\partial \mathbf{z}_{ft}} &= \frac{\partial \mathbf{f}_t}{\partial \mathbf{z}_{ft}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{f}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{f}_t) \circ \mathbf{f}_t \circ \mathbf{C}_{t-1} \circ \frac{\partial C}{\partial \mathbf{C}_t}, \\
\frac{\partial C}{\partial \mathbf{z}_{it}} &= \frac{\partial \mathbf{i}_t}{\partial \mathbf{z}_{it}} \frac{\partial \mathbf{C}_t}{\partial \mathbf{i}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \mathbf{i}_t) \circ \mathbf{i}_t \circ \widetilde{\mathbf{C}}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}, \\
\frac{\partial C}{\partial \mathbf{z}_{ht}} &= \frac{\partial \mathbf{m}_t}{\partial \mathbf{z}_{ht}} \frac{\partial \mathbf{h}_t}{\partial \mathbf{m}_t} \frac{\partial C}{\partial \mathbf{h}_t} = (\mathbf{e} - \mathbf{m}_t) \circ \mathbf{m}_t \circ \mathsf{TanH}(\mathbf{C}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t}, \\
\frac{\partial C}{\partial \mathbf{z}_{Ct}} &= \frac{\partial \widetilde{\mathbf{C}}_t}{\partial \mathbf{z}_{Ct}} \frac{\partial \mathbf{C}_t}{\partial \widetilde{\mathbf{C}}_t} \frac{\partial C}{\partial \mathbf{C}_t} = (\mathbf{e} - \widetilde{\mathbf{C}}_t \circ \widetilde{\mathbf{C}}_t) \circ \mathbf{i}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}.
\end{aligned}$$

BPTT for LSTM

Now we can compute the missing part that will be handed to the previous time step,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_h^\mathsf{T} \frac{\partial C}{\partial \mathbf{z}_t},$$

and the change in the input (handed to the previous layer)

$$\frac{\partial C}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}_t} \frac{\partial C}{\partial \mathbf{z}_t} = \mathbf{W}_x^\mathsf{T} \frac{\partial C}{\partial \mathbf{z}_t}.$$

The update of weights and biases (shared over time) is given by

$$\frac{\partial C}{\partial \mathbf{W}_h} = \sum_{t=0}^{k} \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{h}_{t-1}^\mathsf{T}, \quad \frac{\partial C}{\partial \mathbf{W}_x} = \sum_{t=0}^{k} \frac{\partial C}{\partial \mathbf{z}_t} \mathbf{x}_t^\mathsf{T}, \quad \frac{\partial C}{\partial \mathbf{b}} = \sum_{t=0}^{k} \frac{\partial C}{\partial \mathbf{z}_t}.$$

BPTT for LSTM

The implementation of BPTT for LSTM is based on transferring the state given by

$$\mathbf{W}_h^\mathsf{T} \frac{\partial C}{\partial \mathbf{z}_t}, \quad \mathbf{f}_t \circ \frac{\partial C}{\partial \mathbf{C}_t}$$

to the previous step and computing all partial derivatives for the current time step, updating the derivatives of the weights and biases.

As optimizers all variants of SGD can be used, e.g., Adam.

A list of examples that show results obtained using LSTM can be found at Andrej Karpathy's Blog:

http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Outline

Networks we already know

We already know the following types of neural networks:

Fully Connected Neural Networks: dense layers are activated by various activation functions

Convolutional Neural Networks: convolutional layers replace some/all of the dense layers; weights are shared over spatial coordinates

Recurrent Neural Networks: the fixed inputs/outputs are replaced by sequences of arbitrary length; weights are shared over time

Networks we already know

We already know the following types of neural networks:

Fully Connected Neural Networks: dense layers are activated by various activation functions

Convolutional Neural Networks: convolutional layers replace some/all of the dense layers; weights are shared over spatial coordinates

Recurrent Neural Networks: the fixed inputs/outputs are replaced by sequences of arbitrary length; weights are shared over time

A common theme in all these types of neural networks is to go deeper, i.e., to add more layers and/or to use a larger unroll parameter. This results in richer features, a better feature extraction.

Networks we already know

We already know the following types of neural networks:

Fully Connected Neural Networks: dense layers are activated by various activation functions

Convolutional Neural Networks: convolutional layers replace some/all of the dense layers; weights are shared over spatial coordinates

Recurrent Neural Networks: the fixed inputs/outputs are replaced by sequences of arbitrary length; weights are shared over time

A common theme in all these types of neural networks is to go deeper, i.e., to add more layers and/or to use a larger unroll parameter. This results in richer features, a better feature extraction.

Unfortunately, this does not work out of the box.

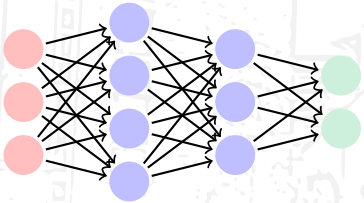
Can we go deeper?

When we use one of the net structures we have and add layers, the training takes longer, the learning rate has to be decreased, and the final accuracy will not be as good as for less layers.

Can we go deeper?

When we use one of the net structures we have and add layers, the training takes longer, the learning rate has to be decreased, and the final accuracy will not be as good as for less layers.

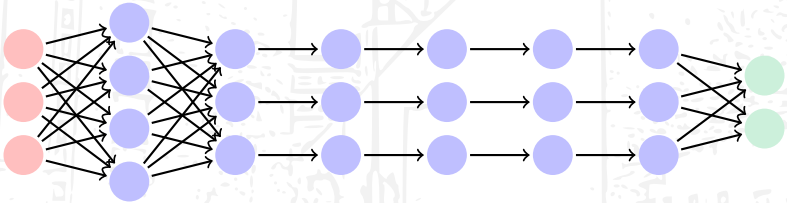
Theoretically, adding new layers at the end of an existing network should not result in a worse approximation, e.g., for ReLU and other variants we could always use the identity transformation and a zero bias vector to extend the net as far as we wish to.



Can we go deeper?

When we use one of the net structures we have and add layers, the training takes longer, the learning rate has to be decreased, and the final accuracy will not be as good as for less layers.

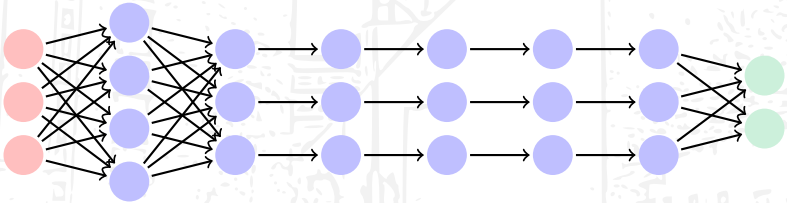
Theoretically, adding new layers at the end of an existing network should not result in a worse approximation, e.g., for ReLU and other variants we could always use the identity transformation and a zero bias vector to extend the net as far as we wish to.



Can we go deeper?

When we use one of the net structures we have and add layers, the training takes longer, the learning rate has to be decreased, and the final accuracy will not be as good as for less layers.

Theoretically, adding new layers at the end of an existing network should not result in a worse approximation, e.g., for ReLU and other variants we could always use the identity transformation and a zero bias vector to extend the net as far as we wish to.



Thus, a trained net of that form should at least be as good as the smaller one.

The problem

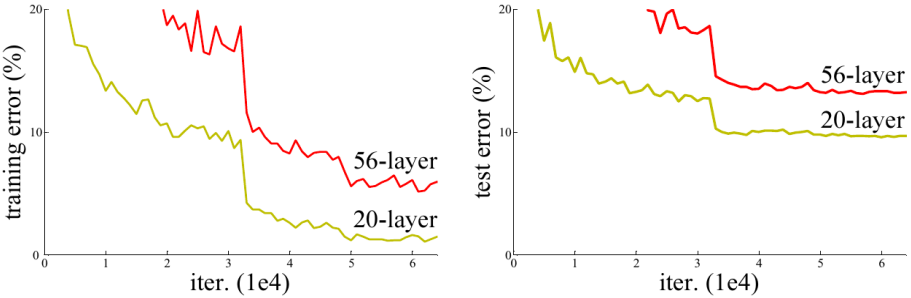


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Image taken from He et al. [3].

Where the problem occurs

Amount of layers in networks:

A few hidden layers: suitable choosen initialization, learning rate, minibatch size mostly suffice

Tens of layers: additionally batch normalization, weight decay, or other regularization has to be used to address the problem of vanishing/exploding gradients

Hundreds to thousands of layers: we have no chance to come up with a satisfactorily trained net using the approaches treated thus far

Where the problem occurs

Amount of layers in networks:

A few hidden layers: suitable choosen initialization, learning rate, minibatch size mostly suffice

Tens of layers: additionally batch normalization, weight decay, or other regularization has to be used to address the problem of vanishing/exploding gradients

Hundreds to thousands of layers: we have no chance to come up with a satisfactorily trained net using the approaches treated thus far

Nets with hundreds of layers trainable by backpropagation using SGD exist. How do they look like?

Ideas to cope with the problem

In LSTM gates enable to keep information over long periods of steps. The idea of using gates can be adapted to feedforward and convolutional networks, see Srivastava, Greff, and Schmidhuber [8] (the extended version is Srivastava, Greff, and Schmidhuber [9]):

$\rightsquigarrow$ Highway Networks

Ideas to cope with the problem

In LSTM gates enable to keep information over long periods of steps. The idea of using gates can be adapted to feedforward and convolutional networks, see Srivastava, Greff, and Schmidhuber [8] (the extended version is Srivastava, Greff, and Schmidhuber [9]):

$\rightsquigarrow$ Highway Networks

We have seen that later layers may at first be set equal to the identity mapping, so we might use the identity transformation as a starting point. This type of view is pursued in He et al. [3]:

$\rightsquigarrow$ Residual Neural Networks (ResNN, also denoted as ResNet)

Ideas to cope with the problem

In LSTM gates enable to keep information over long periods of steps. The idea of using gates can be adapted to feedforward and convolutional networks, see Srivastava, Greff, and Schmidhuber [8] (the extended version is Srivastava, Greff, and Schmidhuber [9]):

$\rightsquigarrow$ Highway Networks

We have seen that later layers may at first be set equal to the identity mapping, so we might use the identity transformation as a starting point. This type of view is pursued in He et al. [3]:

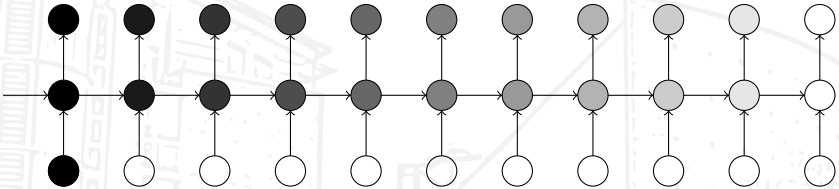
$\rightsquigarrow$ Residual Neural Networks (ResNN, also denoted as ResNet)

> *"Yes, there are two paths you can go by, but in the long run, there's still time to change the road you're on."*

(Led Zeppelin, Stairway to Heaven, 8. November 1971)

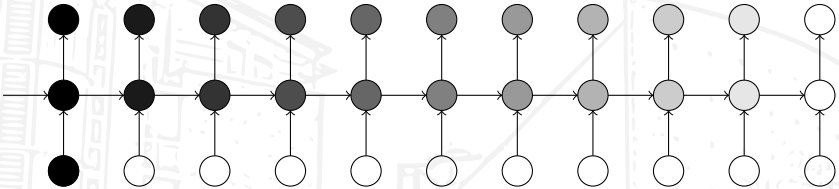
Gates in feedforward/convolutional nets

In the PhD thesis of Graves [2] LSTM are explained as follows. A simple RNN gradually looses information:

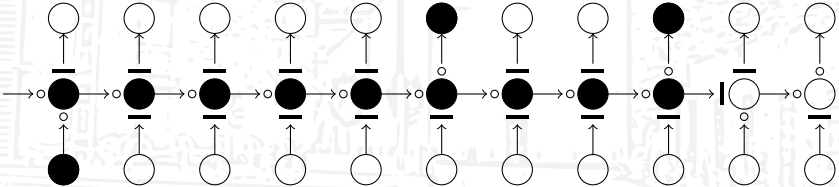


Gates in feedforward/convolutional nets

In the PhD thesis of Graves [2] LSTM are explained as follows. A simple RNN gradually looses information:



An LSTM can keep the information longer due to the three gates:



Gates in feedforward/convolutional nets

Inspired by the gates in LSTM, Srivastava, Greff, and Schmidhuber [8] regulate the information flow in a neural network via a learned gating mechanism.

Gates in feedforward/convolutional nets

Inspired by the gates in LSTM, Srivastava, Greff, and Schmidhuber [8] regulate the information flow in a neural network via a learned gating mechanism. The neural network “can have paths along which information can flow across several layers without attenuation.” These paths are termed Information Highways and the resulting networks Highway Networks.

Gates in feedforward/convolutional nets

Inspired by the gates in LSTM, Srivastava, Greff, and Schmidhuber [8] regulate the information flow in a neural network via a learned gating mechanism. The neural network "can have paths along which information can flow across several layers without attenuation." These paths are termed Information Highways and the resulting networks Highway Networks.

In Highway Networks we replace a layer H that maps input to output of equal dimension, e.g.,

$$\mathbf{y} = H(\mathbf{x}) = a(\mathbf{W}\mathbf{x} + \mathbf{b}), \qquad \mathbf{x}, \mathbf{y} \in \mathbb{R}^n,$$

Gates in feedforward/convolutional nets

Inspired by the gates in LSTM, Srivastava, Greff, and Schmidhuber [8] regulate the information flow in a neural network via a learned gating mechanism. The neural network "can have paths along which information can flow across several layers without attenuation." These paths are termed Information Highways and the resulting networks Highway Networks.

In Highway Networks we replace a layer H that maps input to output of equal dimension, e.g.,

$$\mathbf{y} = H(\mathbf{x}) = a(\mathbf{W}\mathbf{x} + \mathbf{b}), \qquad \mathbf{x}, \mathbf{y} \in \mathbb{R}^n,$$

by including a transform gate T and a carry gate C (often these are coupled, $C = \mathbf{e} - T$),

$$T(\mathbf{x}) = \sigma(\mathbf{W}_t\mathbf{x} + \mathbf{b}_t), \quad C(\mathbf{x}) = \sigma(\mathbf{W}_c\mathbf{x} + \mathbf{b}_c),$$

to obtain, similar to the cell state update in an LSTM, the mapping

$$\mathbf{y} = H(\mathbf{x}) \circ T(\mathbf{x}) + \mathbf{x} \circ C(\mathbf{x}).$$

Results for Highway Networks

They performed experiments that showed “that highway networks as deep as 900 layers can be optimized using simple SGD with momentum.”

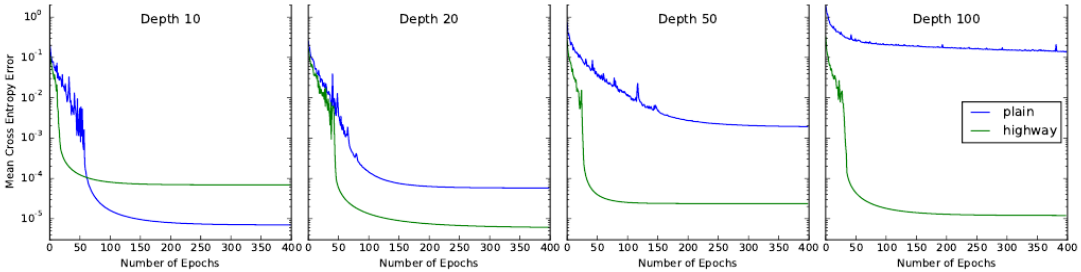


Figure 1. Comparison of optimization of plain networks and highway networks of various depths. All networks were optimized using SGD with momentum. The curves shown are for the best hyperparameter settings obtained for each configuration using a random search. Plain networks become much harder to optimize with increasing depth, while highway networks with up to 100 layers can still be optimized well.

Image taken from Srivastava, Greff, and Schmidhuber [8].

Results for Highway Networks ($C = \mathbf{e} - T$)

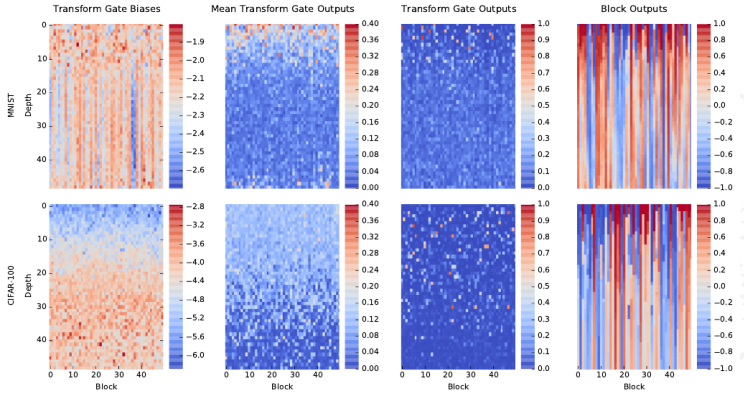


Figure 2. Visualization of certain internals of the blocks in the best 50 hidden layer highway networks trained on MNIST (top row) and CIFAR-100 (bottom row). The first hidden layer is a plain layer which changes the dimensionality of the representation to 50. Each of the 49 highway layers (y-axis) consists of 50 blocks (x-axis). The first column shows the transform gate biases, which were initialized to -2 and -4 respectively. In the second column the mean output of the transform gate over 10,000 training examples is depicted. The third and forth columns show the output of the transform gates and the block outputs for a single random training sample.

Image taken from Srivastava, Greff, and Schmidhuber [8].

Results for Highway Networks ($C = \mathbf{e} - T$)

Initially, the bias of the transformation gate is set to a negative number such that the gate is closed and the carry gate is open, here -2 for the MNIST set and -4 for the CIFAR-100 set. This ensures that in the beginning the data flows through the net mostly unchanged.

Results for Highway Networks ($C = \mathbf{e} - T$)

Initially, the bias of the transformation gate is set to a negative number such that the gate is closed and the carry gate is open, here -2 for the MNIST set and -4 for the CIFAR-100 set. This ensures that in the beginning the data flows through the net mostly unchanged.

To investigate how much this changed after training, the authors changed the net by closing the transformation gates of one layer and compared the performance of this net to the original one. They call this lesioning a layer.

Results for Highway Networks ($C = \mathbf{e} - T$)

Initially, the bias of the transformation gate is set to a negative number such that the gate is closed and the carry gate is open, here -2 for the MNIST set and -4 for the CIFAR-100 set. This ensures that in the beginning the data flows through the net mostly unchanged.

To investigate how much this changed after training, the authors changed the net by closing the transformation gates of one layer and compared the performance of this net to the original one. They call this lesioning a layer.

In MNIST, lesioning the layers 15–45 seems to have no effect (MNIST is very simple dataset). In CIFAR-100 the effect is more dramatic in earlier layers; the effect decays linearly in the layer number.

Results for Highway Networks ($C = \mathbf{e} - T$)

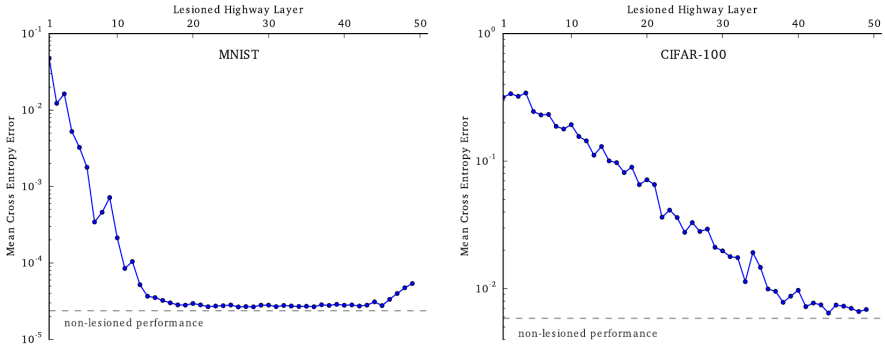


Figure 4: Lesioned training set performance (y-axis) of the best 50-layer highway networks on MNIST (left) and CIFAR-100 (right), as a function of the lesioned layer (x-axis). Evaluated on the full training set while forcefully closing all the transform gates of a single layer at a time. The non-lesioned performance is indicated as a dashed line at the bottom.

Image taken from Srivastava, Greff, and Schmidhuber [9].

Approximating the identity

In He et al. [3] the authors pursue the idea that it is easier to approximate a residual than the function,

Approximating the identity

In He et al. [3] the authors pursue the idea that it is easier to approximate a residual than the function, e.g., for a single layer, in place of approximating $\mathcal{H} \approx \mathrm{id}$, they approximate $\mathcal{F}$:

$$\mathbf{y} = \mathcal{H}(\mathbf{x}) \approx \widehat{a}(\widehat{\mathbf{W}}\mathbf{x} + \widehat{\mathbf{b}}) \quad \rightsquigarrow \quad \mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \approx a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

Approximating the identity

In He et al. [3] the authors pursue the idea that it is easier to approximate a residual than the function, e.g., for a single layer, in place of approximating $\mathcal{H} \approx \mathrm{id}$, they approximate $\mathcal{F}$:

$$\mathbf{y} = \mathcal{H}(\mathbf{x}) \approx \widehat{a}(\widehat{\mathbf{W}}\mathbf{x} + \widehat{\mathbf{b}}) \quad \rightsquigarrow \quad \mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \approx a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

This Residual Net (ResNet, ResNN) is a special case of a Highway Network ($T = \mathbf{e}$, $C = \mathbf{e}$, $H = \mathcal{F}$).

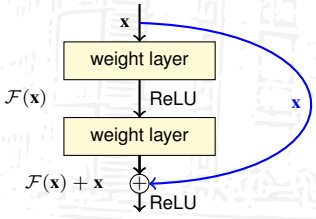
Approximating the identity

In He et al. [3] the authors pursue the idea that it is easier to approximate a residual than the function, e.g., for a single layer, in place of approximating $\mathcal{H} \approx \mathrm{id}$, they approximate $\mathcal{F}$:

$$\mathbf{y} = \mathcal{H}(\mathbf{x}) \approx \widehat{a}(\widehat{\mathbf{W}}\mathbf{x} + \widehat{\mathbf{b}}) \quad \rightsquigarrow \quad \mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \approx a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

This Residual Net (ResNet, ResNN) is a special case of a Highway Network ($T = \mathbf{e}$, $C = \mathbf{e}$, $H = \mathcal{F}$).

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ is realized by a feedforward neural network with shortcut connections:



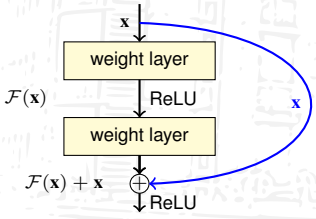
Approximating the identity

In He et al. [3] the authors pursue the idea that it is easier to approximate a residual than the function, e.g., for a single layer, in place of approximating $\mathcal{H} \approx \mathrm{id}$, they approximate $\mathcal{F}$:

$$\mathbf{y} = \mathcal{H}(\mathbf{x}) \approx \widehat{a}(\widehat{\mathbf{W}}\mathbf{x} + \widehat{\mathbf{b}}) \quad \rightsquigarrow \quad \mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \approx a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

This Residual Net (ResNet, ResNN) is a special case of a Highway Network ($T = \mathbf{e}$, $C = \mathbf{e}$, $H = \mathcal{F}$).

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ is realized by a feedforward neural network with shortcut connections:



In this example, which is used in He et al. [3],

$$\mathcal{F}(\mathbf{x}) := \mathbf{W}_2 \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

is followed by the addition and then the activation,

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}, \quad \mathbf{z} = \mathrm{ReLU}(\mathbf{y}).$$

Definition of ResNet-34

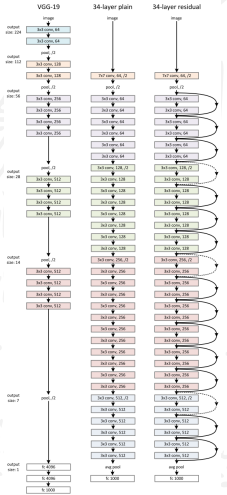
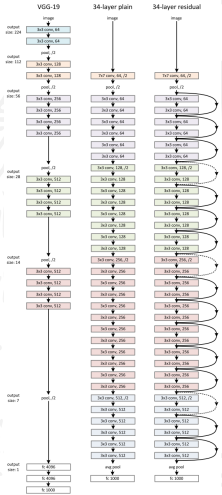


Image taken from He et al. [3].

Definition of ResNet-34



The difference between the plain-34 net and the ResNet-34 in He et al. [3] lies in the additional 16 shortcut connections. For comparison they included in the first column the VGG-19 net with 19 layers.

Image taken from He et al. [3].

Definition of ResNet-34

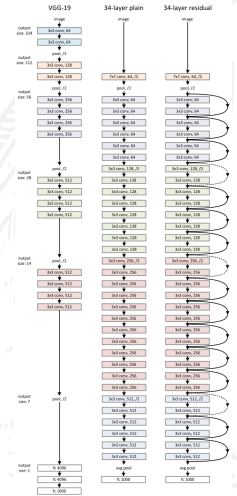


Image taken from He et al. [3].

The difference between the plain-34 net and the ResNet-34 in He et al. [3] lies in the additional 16 shortcut connections. For comparison they included in the first column the VGG-19 net with 19 layers.

The *-34 nets begin with a 7×7 convolution layer, end with a fully connected layer with 1000 neurons and SoftMax activation. All other layers are 3×3 convolution layers, some with stride 2 (pooling).

Definition of ResNet-34

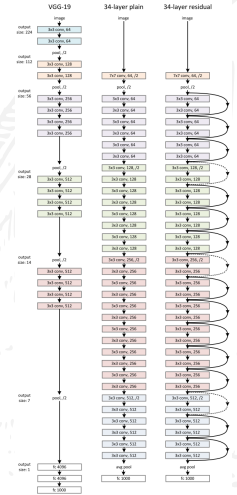


Image taken from He et al. [3].

The difference between the plain-34 net and the ResNet-34 in He et al. [3] lies in the additional 16 shortcut connections. For comparison they included in the first column the VGG-19 net with 19 layers.

The *-34 nets begin with a 7×7 convolution layer, end with a fully connected layer with 1000 neurons and SoftMax activation. All other layers are 3×3 convolution layers, some with stride 2 (pooling).

Three of the shortcut connections are based on padding the input by zeros, as the dimension increases, these are the ones with the dashed arrows in the picture. Such transformations have already been defined in Srivastava, Greff, and Schmidhuber [9].

Results for ResNet-{18,34}

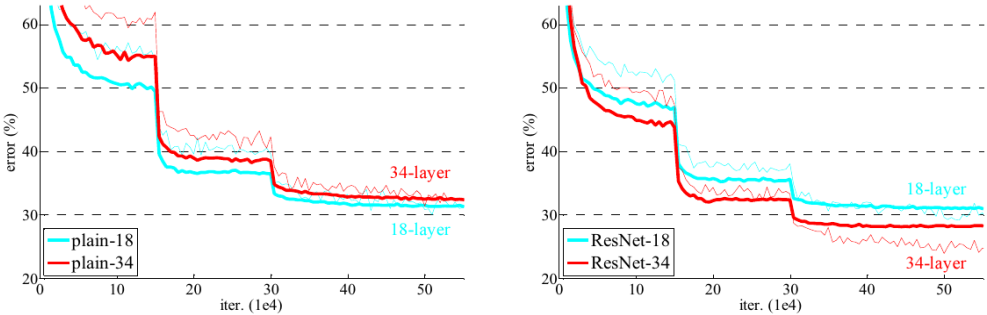


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Image taken from He et al. [3].

Variants of ResNet (scramble BN, ReLU, affine linear)

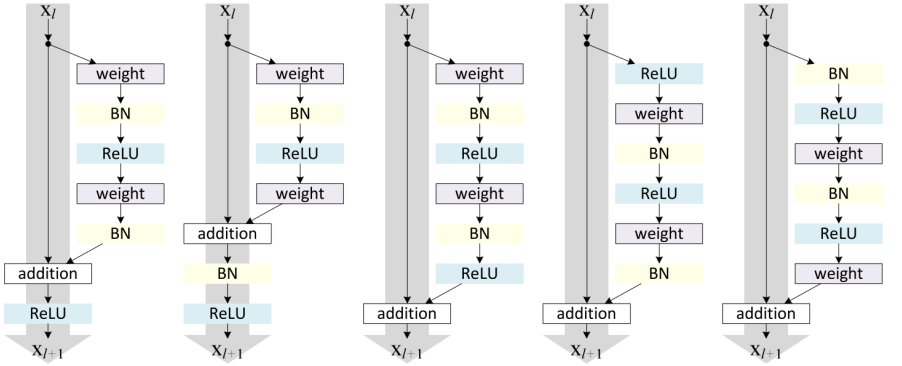


Image taken from He et al. [4].

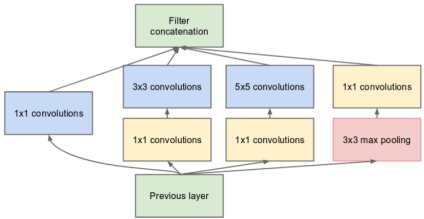
ResNet-1001, ResNeXt

Based on the full pre-activation variant, He et al. [4] define a ResNet-1001 with 1001 layers that they successfully trained.

ResNet-1001, ResNeXt

Based on the full pre-activation variant, He et al. [4] define a ResNet-1001 with 1001 layers that they successfully trained.

ResNeXt: Xie et al. [12] combine ResNet with the Inception modules of GoogLeNet V1 from Szegedy et al. [10],



(b) Inception module with dimension reductions

Image taken from Szegedy et al. [10].

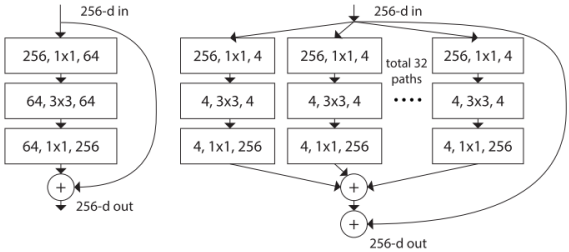


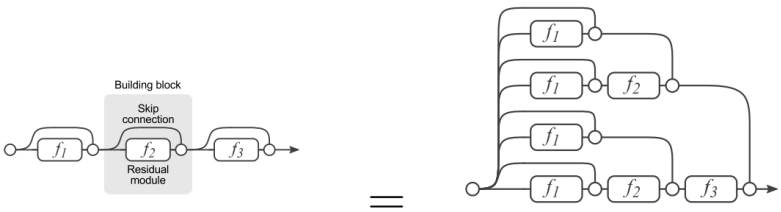
Image taken from Xie et al. [12].

How many paths do we have?

Veit, Wilber, and Belongie [11] show that a ResNet with m layers has 2^m connections; they argue that a ResNet is an ensemble of smaller nets.

How many paths do we have?

Veit, Wilber, and Belongie [11] show that a ResNet with m layers has 2^m connections; they argue that a ResNet is an ensemble of smaller nets.



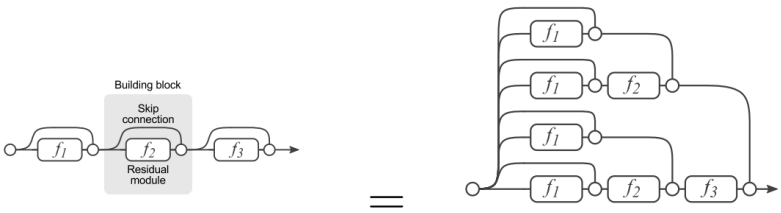
(a) Conventional 3-block residual network

(b) Unraveled view of (a)

Image taken from Veit, Wilber, and Belongie [11].

How many paths do we have?

Veit, Wilber, and Belongie [11] show that a ResNet with m layers has 2^m connections; they argue that a ResNet is an ensemble of smaller nets.



(a) Conventional 3-block residual network (b) Unraveled view of (a)

Image taken from Veit, Wilber, and Belongie [11].

They investigate how removal of a residual block changes the performance. In the next plot the arising lesioning is shown.

How many paths do we need?

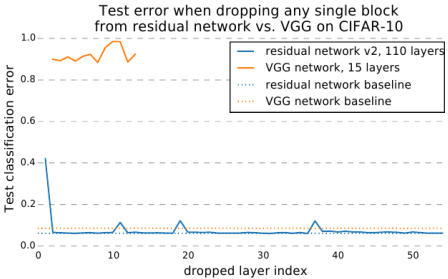


Figure 3: Deleting individual layers from VGG and a residual network on CIFAR-10. VGG performance drops to random chance when any one of its layers is deleted, but deleting individual modules from residual networks has a minimal impact on performance. Removing downsampling modules has a slightly higher impact.

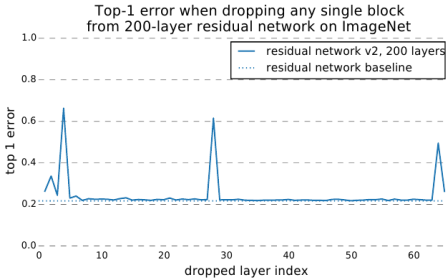


Figure 4: Results when dropping individual blocks from residual networks trained on ImageNet are similar to CIFAR results. However, downsampling layers tend to have more impact on ImageNet.

Image taken from Veit, Wilber, and Belongie [11].

Training w/ stochastic depth

Training a ResNet takes a long time.

Training w/ stochastic depth

Training a ResNet takes a long time.

Similar to Dropout and DropConnect, Huang et al. [6] keep residual blocks based on the probability

$$p_\ell = 1 - \frac{\ell}{L}(1 - p_L), \quad \text{where mostly } p_L = .5$$

in the layer ℓ of the L layers during training. The earlier layers are more important for the feature extraction and are more often kept. The input layer $\ell = 0$ is kept always.

Training w/ stochastic depth

Training a ResNet takes a long time.

Similar to Dropout and DropConnect, Huang et al. [6] keep residual blocks based on the probability

$$p_\ell = 1 - \frac{\ell}{L}(1 - p_L), \quad \text{where mostly } p_L = .5$$

in the layer ℓ of the L layers during training. The earlier layers are more important for the feature extraction and are more often kept. The input layer $\ell = 0$ is kept always.

At test time, all layers are used with the residual block multiplied by p_ℓ,

$$\mathbf{a}_\ell = \text{ReLU}(p_\ell \cdot \mathcal{F}_{\ell-1}(\mathbf{a}_{\ell-1}) + \mathbf{a}_{\ell-1}).$$

According to the authors this speeds up the training w/o spoiling the accuracy.

Training w/ stochastic depth

Training a ResNet takes a long time.

Similar to Dropout and DropConnect, Huang et al. [6] keep residual blocks based on the probability

$$p_\ell = 1 - \frac{\ell}{L}(1 - p_L), \quad \text{where mostly } p_L = .5$$

in the layer ℓ of the L layers during training. The earlier layers are more important for the feature extraction and are more often kept. The input layer $\ell = 0$ is kept always.

At test time, all layers are used with the residual block multiplied by p_ℓ,

$$\mathbf{a}_\ell = \text{ReLU}(p_\ell \cdot \mathcal{F}_{\ell-1}(\mathbf{a}_{\ell-1}) + \mathbf{a}_{\ell-1}).$$

According to the authors this speeds up the training w/o spoiling the accuracy.

They successfully trained a ResNet with 1202 layers based on training w/ stochastic depth.

Adding more paths

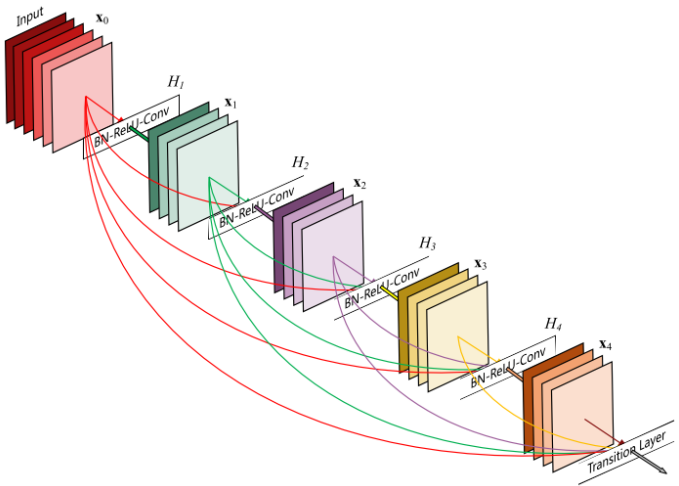


Image taken from Huang et al. [7].

Adding more paths

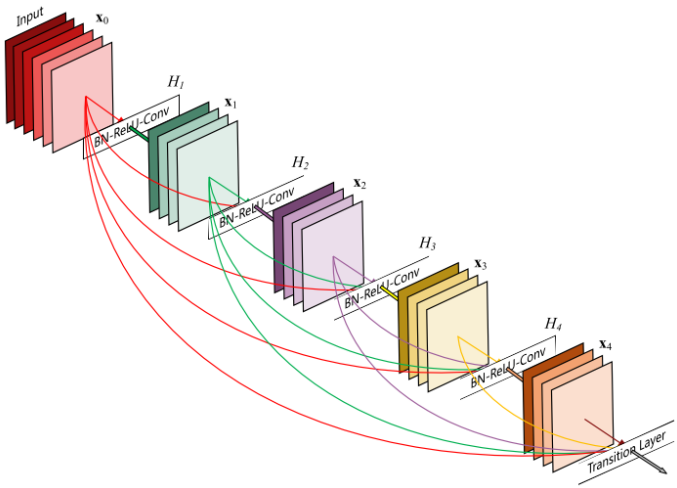


Image taken from Huang et al. [7].

Huang et al. [7] use every feature map ever constructed in every later layer. The resulting net is called a DenseNet.

Adding more paths

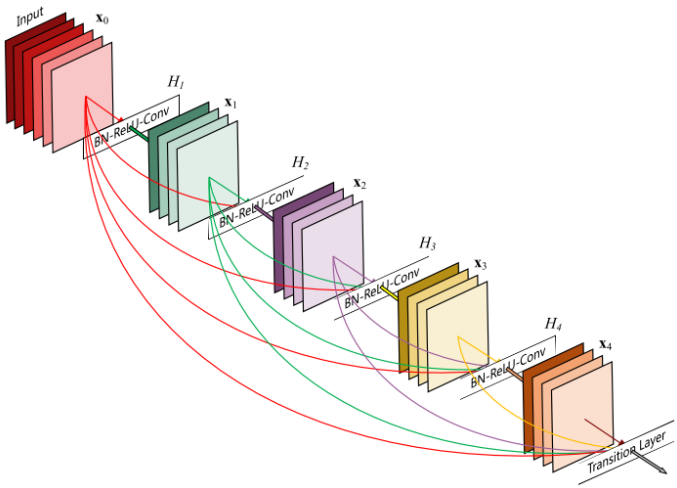


Image taken from Huang et al. [7].

Huang et al. [7] use every feature map ever constructed in every later layer. The resulting net is called a DenseNet.

A DenseNet has

$$\frac{L(L+1)}{2}$$

shortcut connections.

Adding more paths

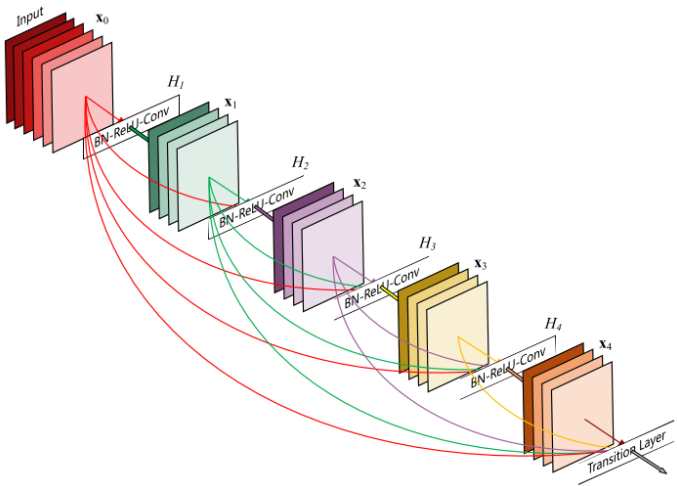


Image taken from Huang et al. [7].

Huang et al. [7] use every feature map ever constructed in every later layer. The resulting net is called a DenseNet.

A DenseNet has

$$\frac{L(L+1)}{2}$$

shortcut connections.

In contrast to a ResNet, the inputs are concatenated.

References I

[1] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: *EMNLP*. ACL, 2014, pp. 1724–1734.

[2] Alex Graves. “Supervised Sequence Labelling with Recurrent Neural Networks”. Dissertation. Technische Universität München, Fakultät für Informatik, 2008. URL: https://www.cs.toronto.edu/~graves/phd.pdf.

[3] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016. DOI: 10.1109/CVPR.2016.90.

[4] Kaiming He et al. “Identity Mappings in Deep Residual Networks”. In: *arXiv e-prints*, arXiv:1603.05027 (Mar. 2016). arXiv: 1603.05027 [cs.CV].

[5] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[6] Gao Huang et al. “Deep Networks with Stochastic Depth”. In: *arXiv e-prints*, arXiv:1603.09382 (Mar. 2016). arXiv: 1603.09382 [cs.LG].

References II

[7] Gao Huang et al. “Densely Connected Convolutional Networks”. In: *arXiv e-prints*, arXiv:1608.06993 (Aug. 2016). arXiv: 1608.06993 [cs.CV].

[8] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. “Highway Networks”. In: *arXiv e-prints*, arXiv:1505.00387 (May 2015). arXiv: 1505.00387 [cs.LG].

[9] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. “Training Very Deep Networks”. In: *arXiv e-prints*, arXiv:1507.06228 (July 2015). published as https://papers.nips.cc/paper/5850-training-very-deep-networks.pdf. arXiv: 1507.06228 [cs.LG].

[10] Christian Szegedy et al. “Going deeper with convolutions”. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2015. DOI: 10.1109/CVPR.2015.7298594.

[11] Andreas Veit, Michael Wilber, and Serge Belongie. “Residual Networks Behave Like Ensembles of Relatively Shallow Networks”. In: *arXiv e-prints*, arXiv:1605.06431 (May 2016). arXiv: 1605.06431 [cs.CV].

References III

[12] Saining Xie et al. “Aggregated Residual Transformations for Deep Neural Networks”. In: *arXiv e-prints*, arXiv:1611.05431 (Nov. 2016). arXiv: 1611.05431 [cs.CV].